Final Project
Multi-Class Texture Analysis in Colorectal Cancer Histology

By: Nicole Poliak

Dataset Information

- **Dataset Name:** Collection of textures in colorectal cancer histology

  Total Data Instances: 5,000 Images

  Features: Image Shape: 150x150x3 (RGB) Pixels (67,500-Dimensional)

  Target Variable: Tissue Label (Encoded as 0 to 7)

  Data Type: RGB Image Data

- **Train/Test Splits:**

  Train Data: 80% of the Dataset (4,000 Images)

  Test Data: 20% of the Dataset (1,000 Images)

- **Preprocessing:**

  Data Loading: Custom Collection

  Data Exploration: Displayed a sample of images with their corresponding labels

  Data Splitting: Split into training and testing sets (80/20)

  Image Resizing Methods: Downsample to 32x32, PCA to 256-D , VGG16 backbone

- **Project Objective:**

  Objective: Classification of Colorectal Histology Images into tissue types

  Business Impact: Medical Diagnosis and Research in Colorectal Health

  Metrics: Accuracy

- **Experiments:**

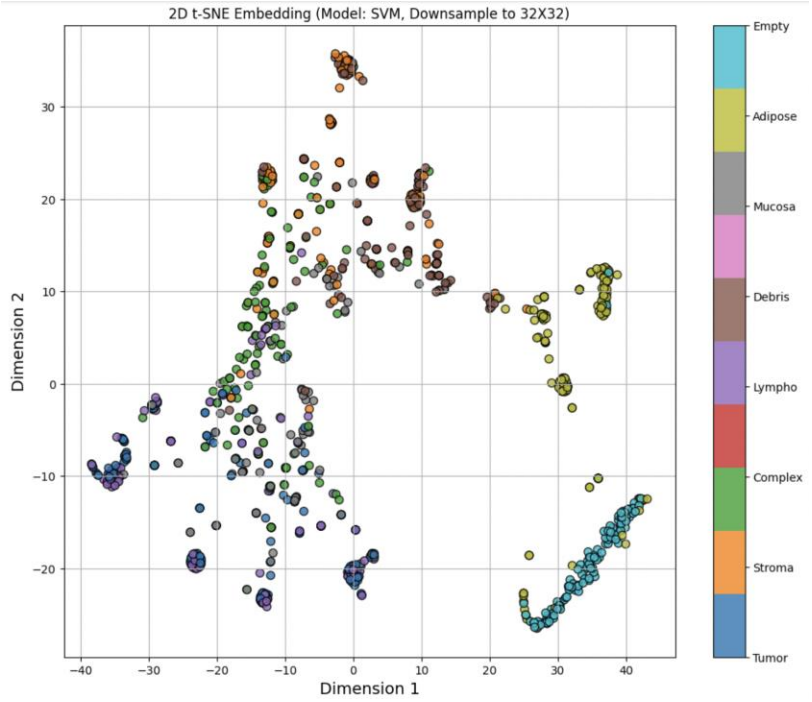
  Total Experiments: 9

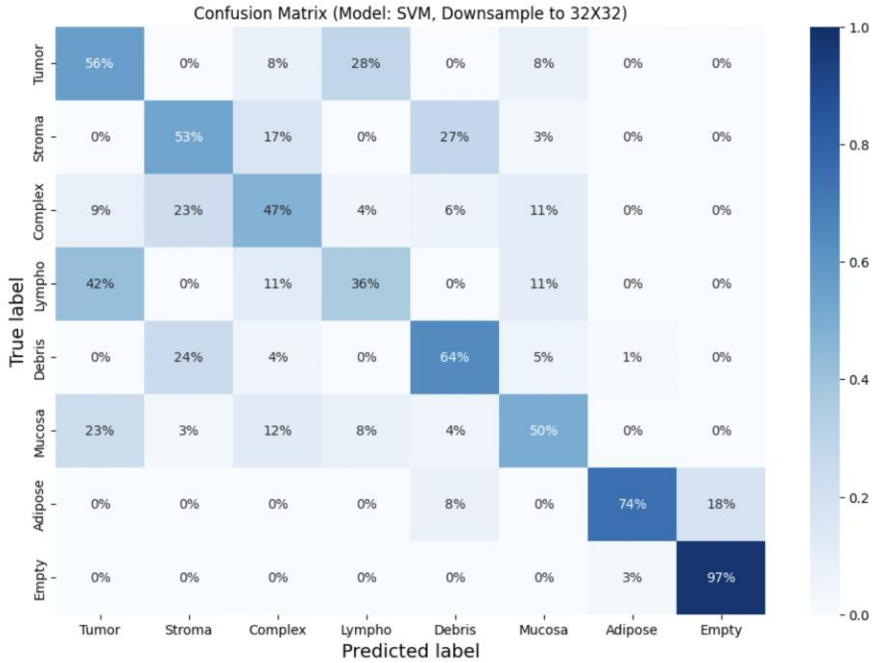
  Methods: Resizing (3) x Classifiers (3)

  Classifiers: SVM, Softmax, Neural Network (softmax+ relu)

Results

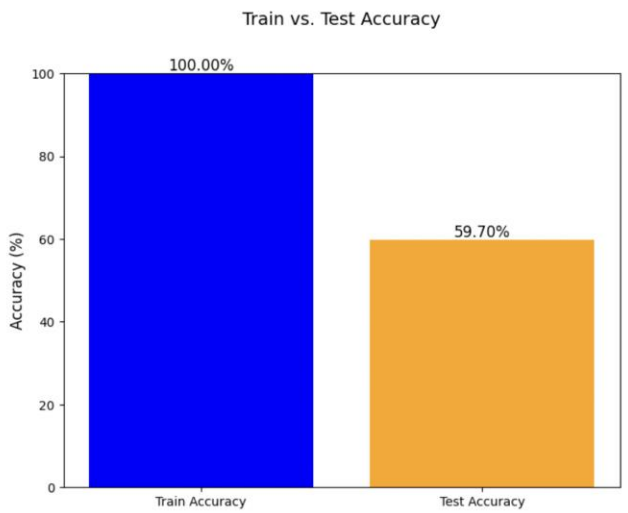
Model 1 – SVM + Downsample to 32x32






```
Train Accuracy: 100.00%
Test Accuracy: 59.70%
```

(*significant overfitting)

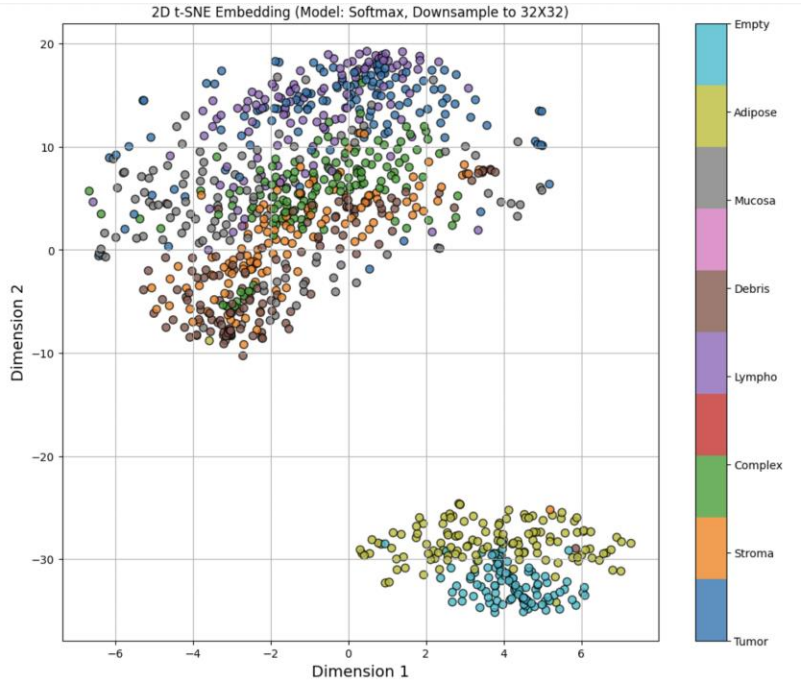


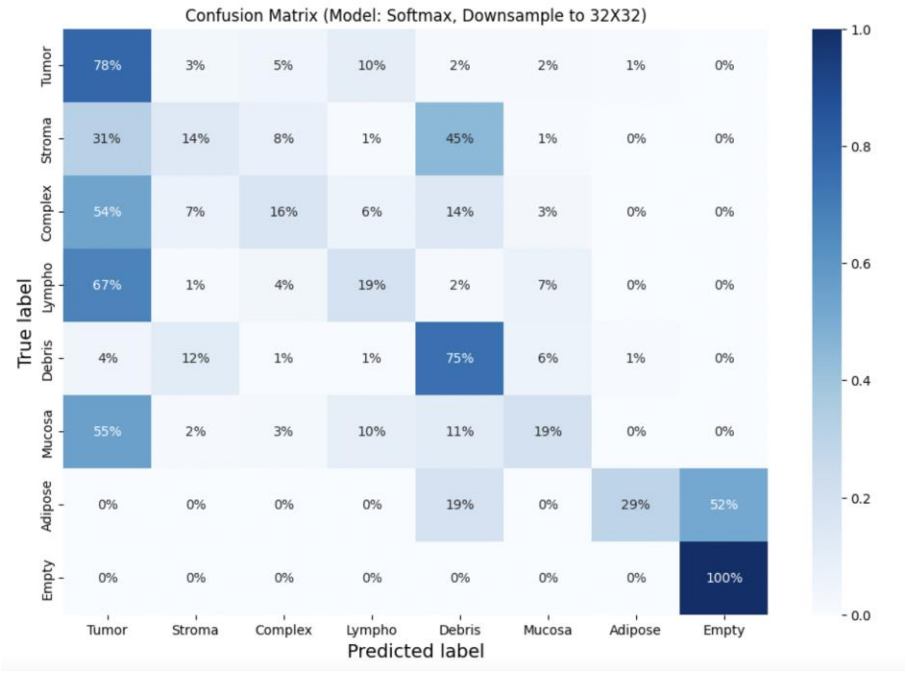
- Dimensionality reduction: downsampling image to 32x32 and then flattening, resulting in a 3072-D feature space.

```
(4000, 3072)
(1000, 3072)
```

- Classifier: SVM classifier (C = 0.01, kernel = 'linear').
- I used GridSearchCV to find the best hyperparameter C.

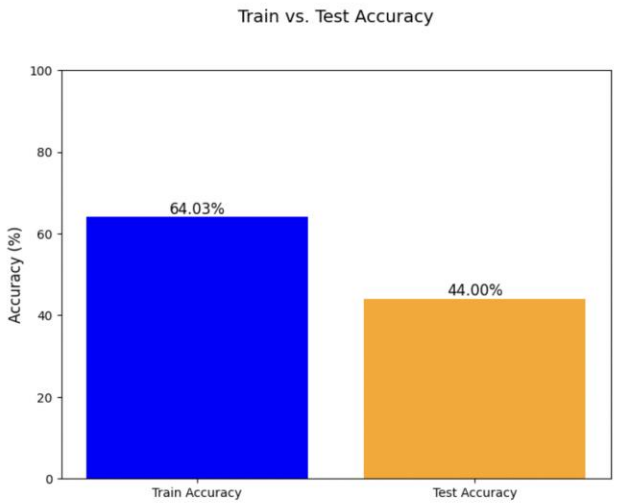
Model 2 – Softmax + Downsample to 32X32





Train Accuracy: 64.03%
Test Accuracy: 44.00%

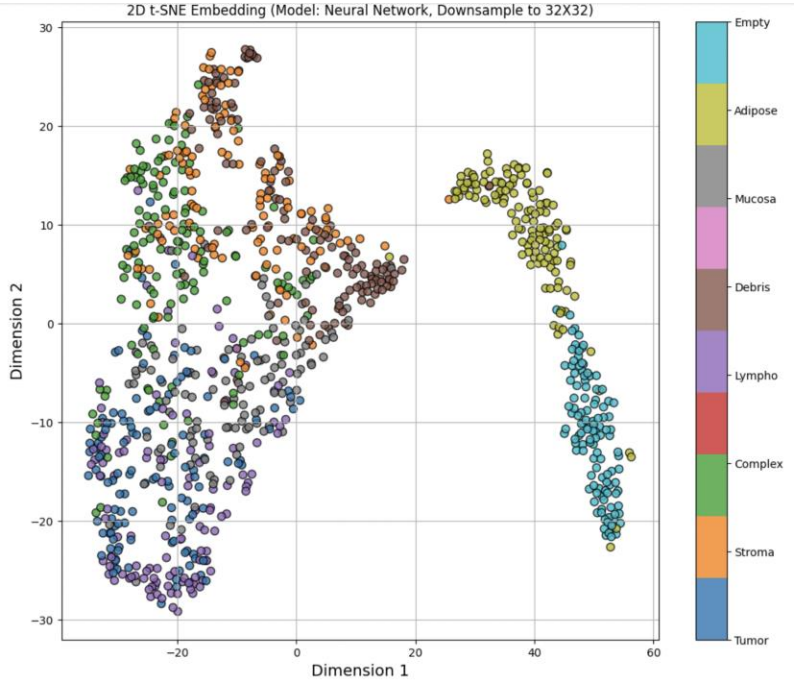
(*significant overfitting)

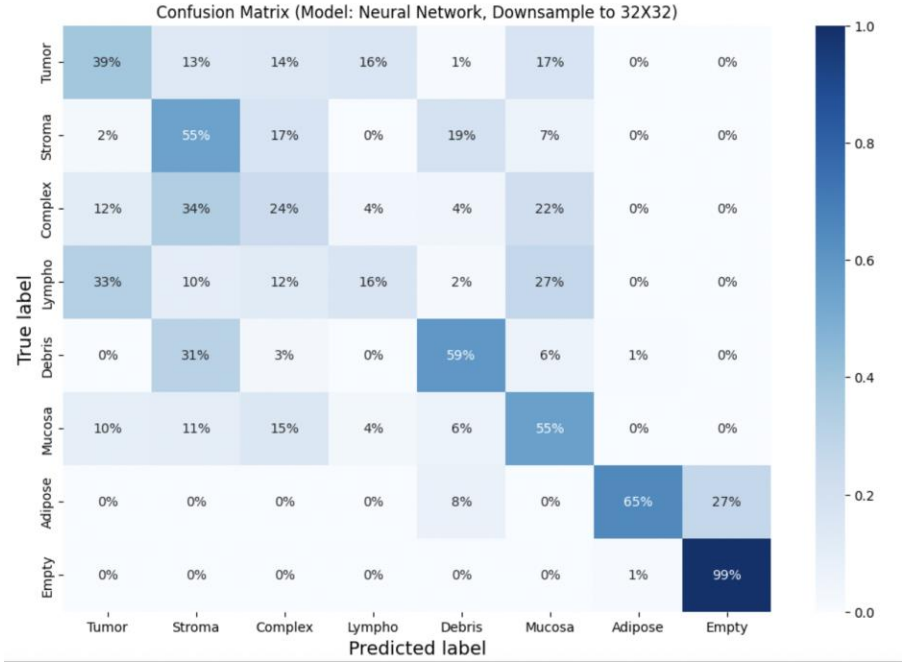


- Dimensionality reduction: downsampling image to 32x32 and then flattening, resulting in a 3072-D feature space.
- Classifier: Softmax classifier (epochs = 50, batch_size = 64).

```
(4000, 3072)
(1000, 3072)
```

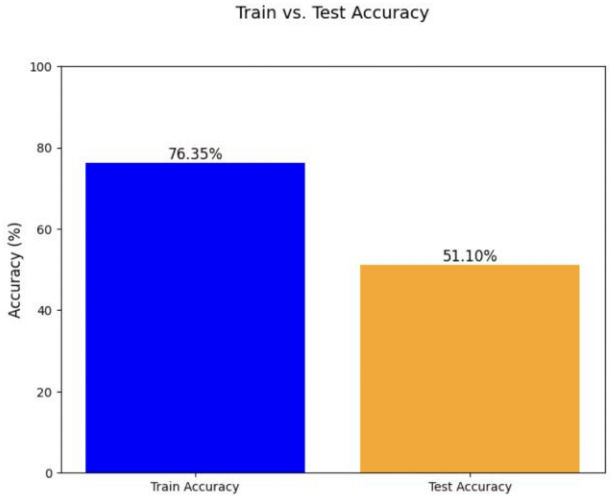
Model 3 – Neural Network + Downsample to 32x32





Train Accuracy: 76.35%
Test Accuracy: 51.10%

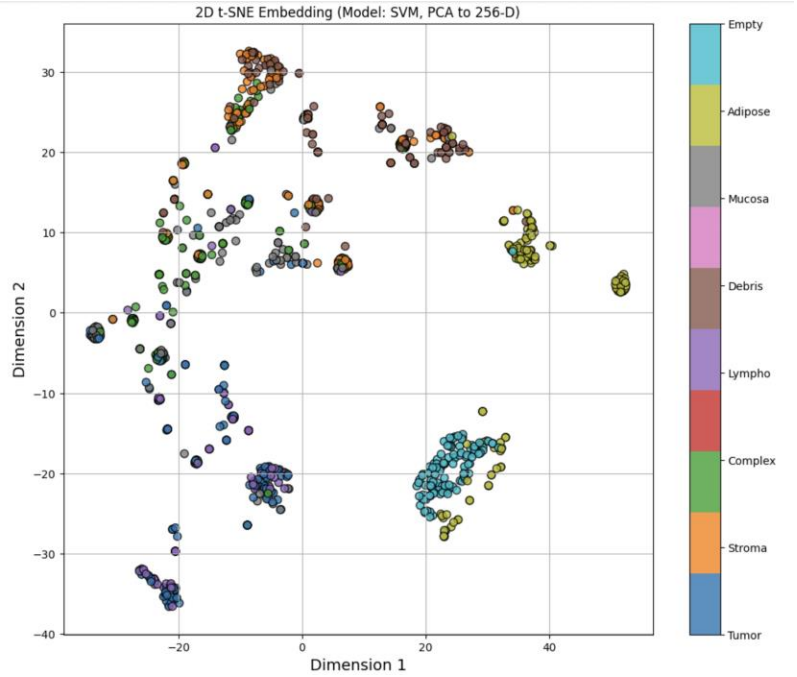
(*significant overfitting)

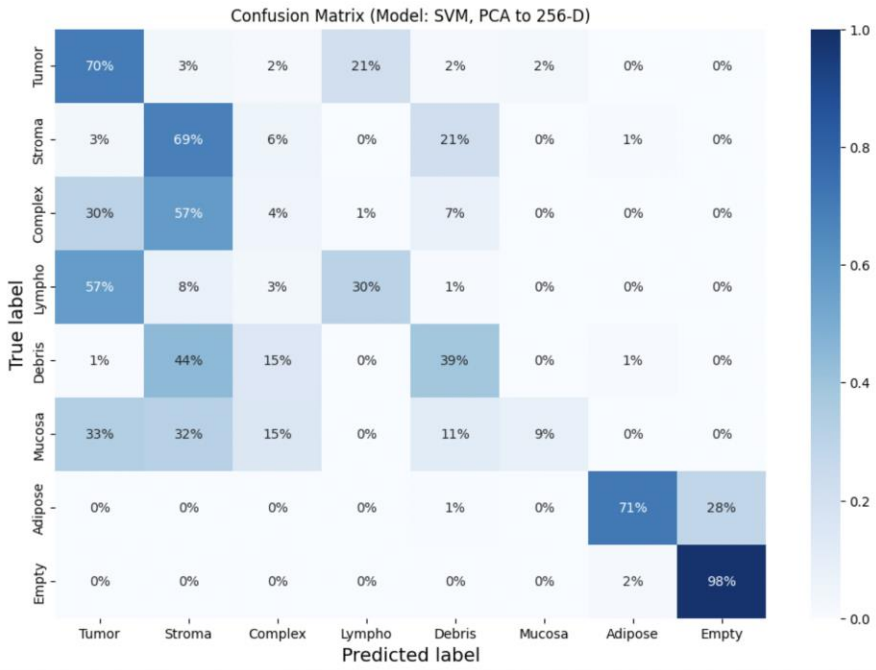


- Dimensionality reduction: downsampling image to 32x32 and then flattening, resulting in a 3072-D feature space.
- Classifier: Neural Network classifier (epochs = 50, batch_size = 64).

```
(4000, 3072)
(1000, 3072)
```

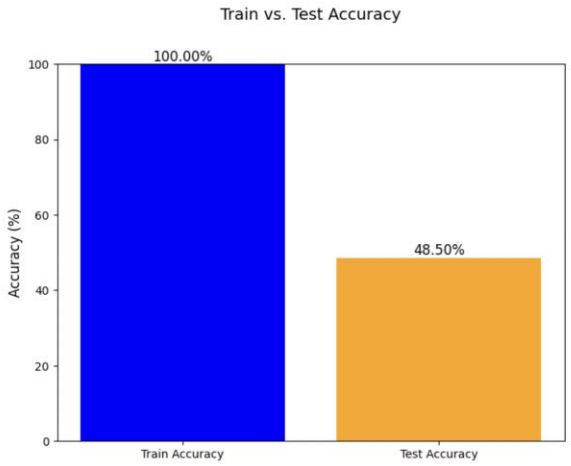
Model 4 – SVM + PCA to 256D






```
Train Accuracy: 100.00%
Test Accuracy: 48.50%
```

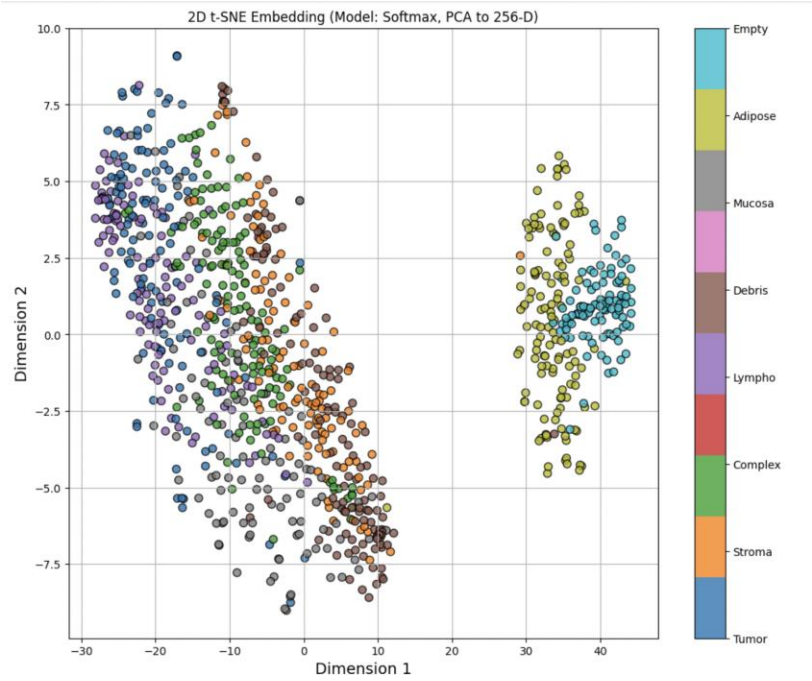
(*significant overfitting)

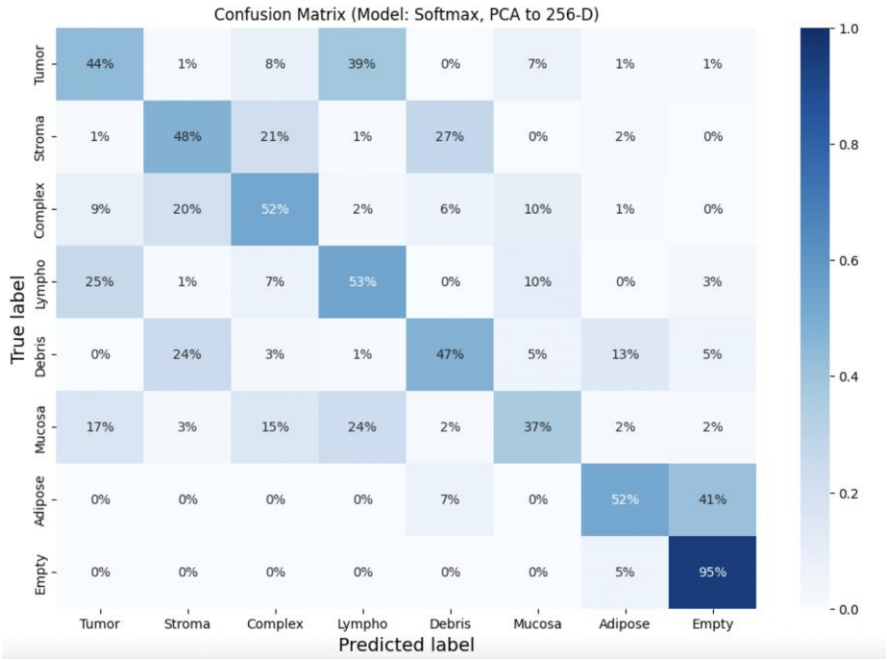


- Dimensionality reduction: PCA to 256D and then flattening, resulting in a 256-D feature space.
- Classifier: SVM classifier (C = 0.01, kernel = 'linear').
- I used batch_size = 40 to perform a mini-batch training.

```
(4000, 256)
(1000, 256)
```

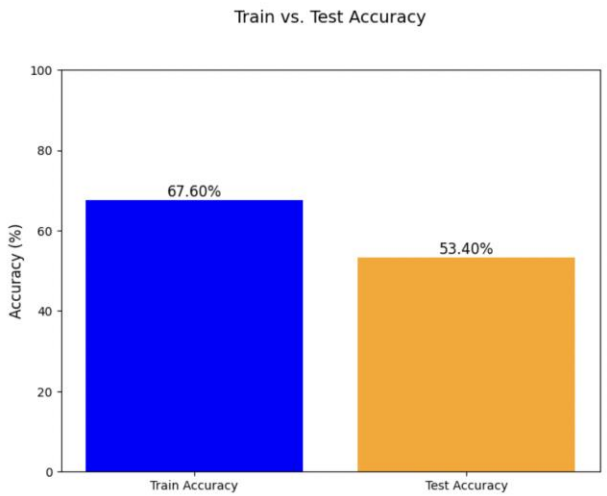
Model 5 – Softmax + PCA to 256D





Train Accuracy: 67.60%
Test Accuracy: 53.40%

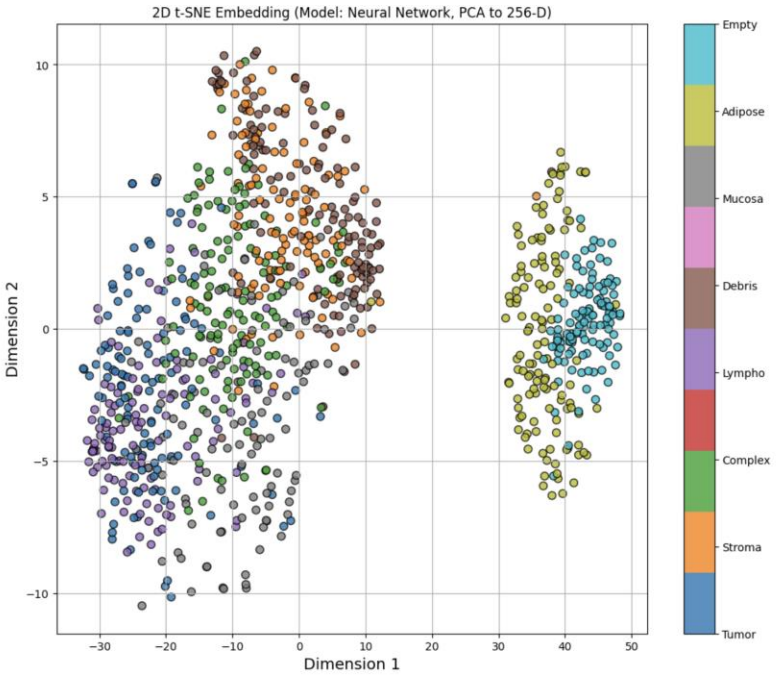
(*overfitting)

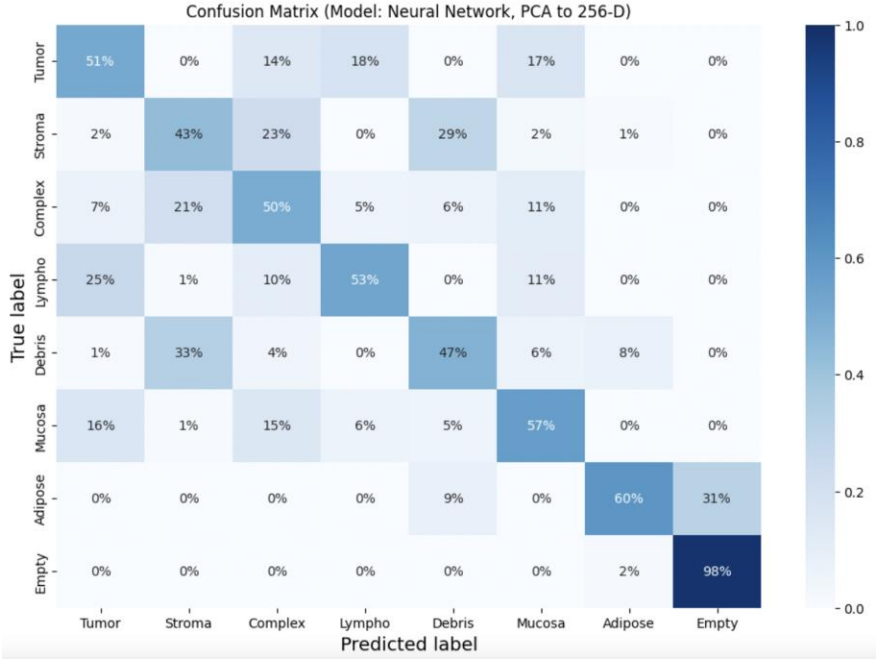


- Dimensionality reduction: PCA to 256D and then flattening, resulting in a 256-D feature space.
- Classifier: Softmax classifier (epochs = 50, batch_size = 64).

```
(4000, 256)
(1000, 256)
```

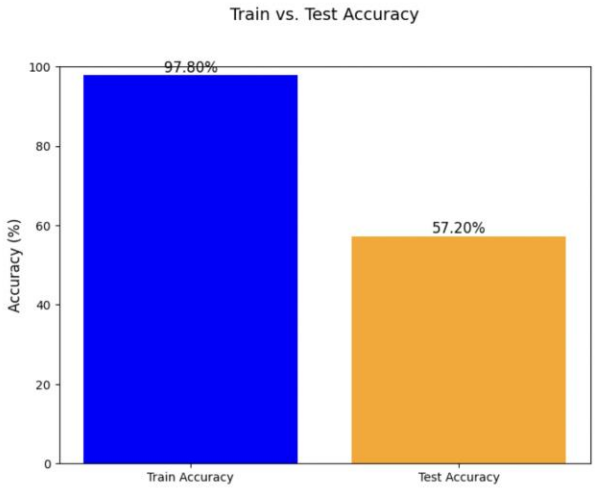
Model 6 – Neural Network + PCA to 256D





Train Accuracy: 97.80%
Test Accuracy: 57.20%

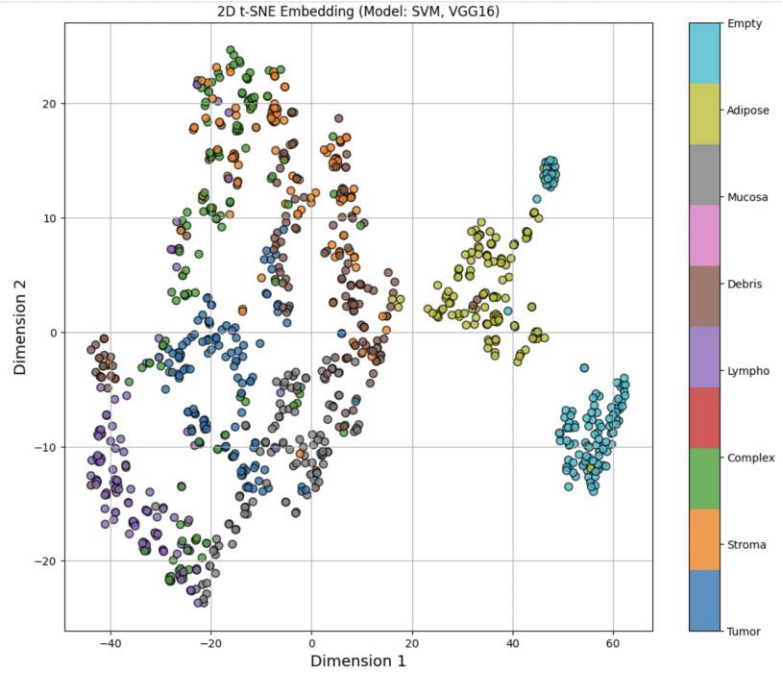
(*significant overfitting)

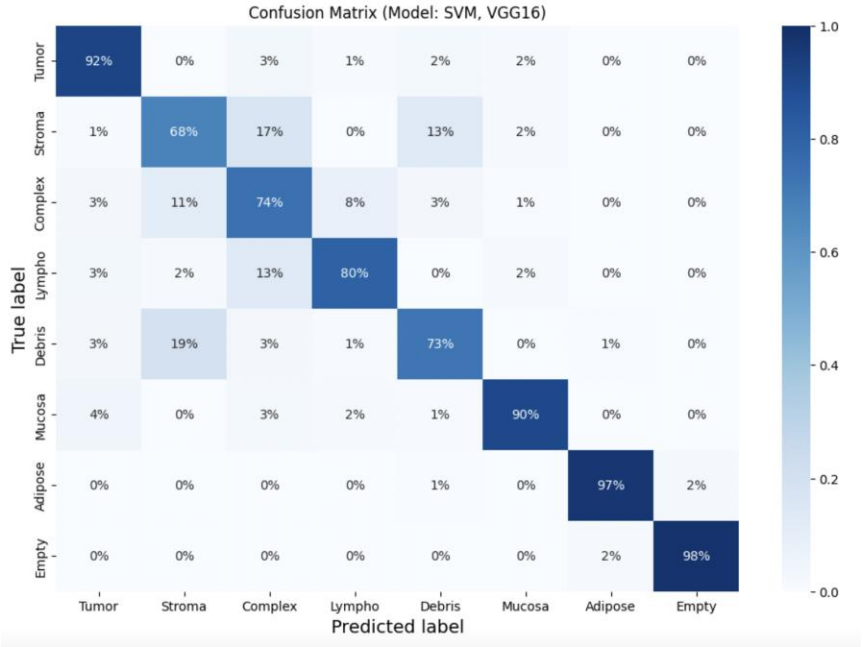


- Dimensionality reduction: PCA to 256D and then flattening, resulting in a 256-D feature space.
- Classifier: Neural Network classifier (epochs = 50, batch_size = 64).

```
(4000, 256)
(1000, 256)
```

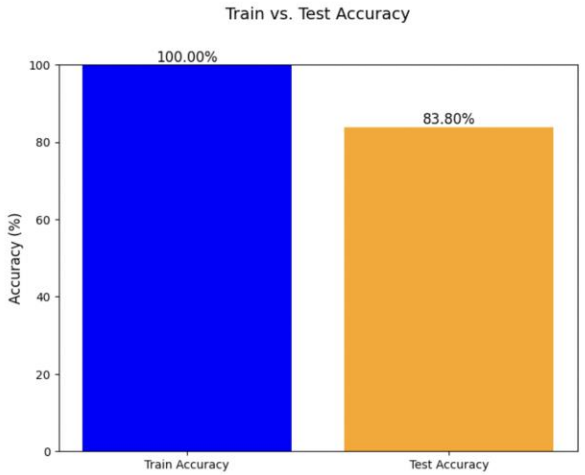
Model 7 – SVM + VGG16






```
Train Accuracy: 100.00%
Test Accuracy: 83.80%
```

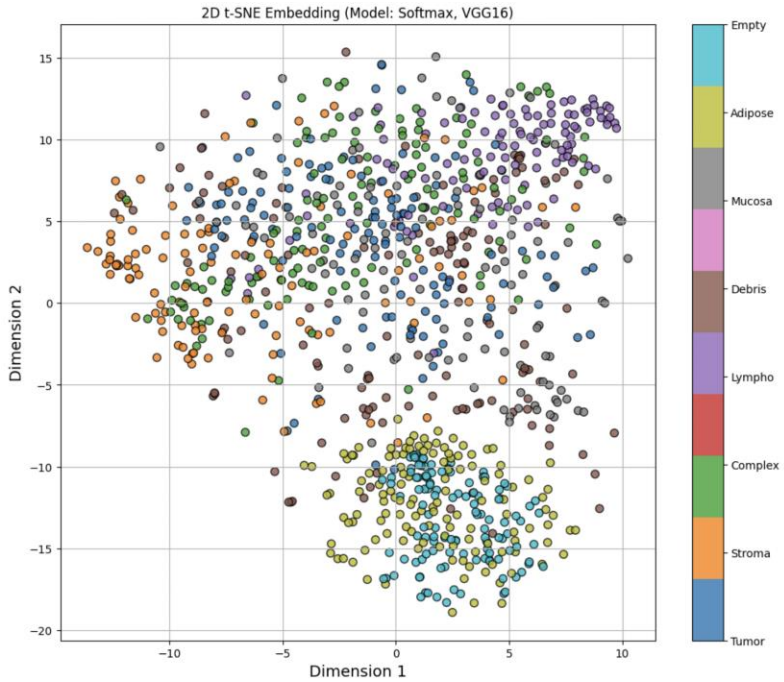
(*overfitting, but good results)

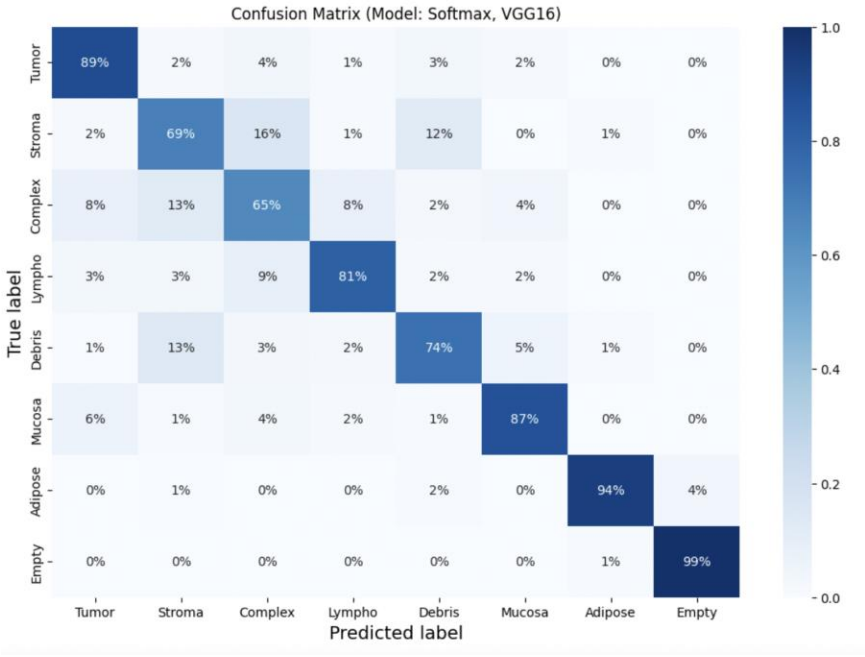


- Dimensionality reduction: VGG16 and then flattening, resulting in a 8192-D feature space.
- Classifier: SVM classifier (C = 0.01, kernel = 'linear').

```
(4000, 8192)
(1000, 8192)
```

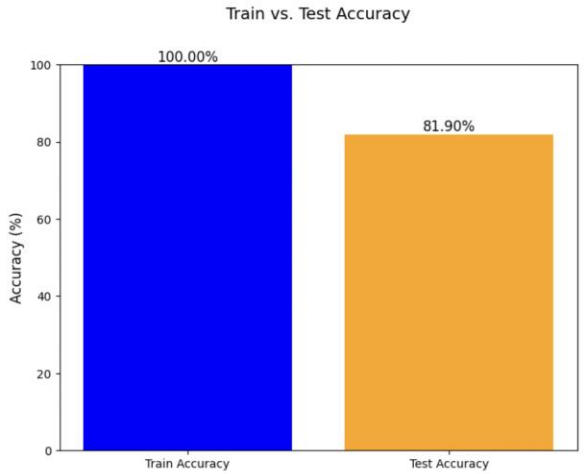
Model 8 – Softmax + VGG16






```
Train Accuracy: 100.00%
Test Accuracy: 81.90%
```

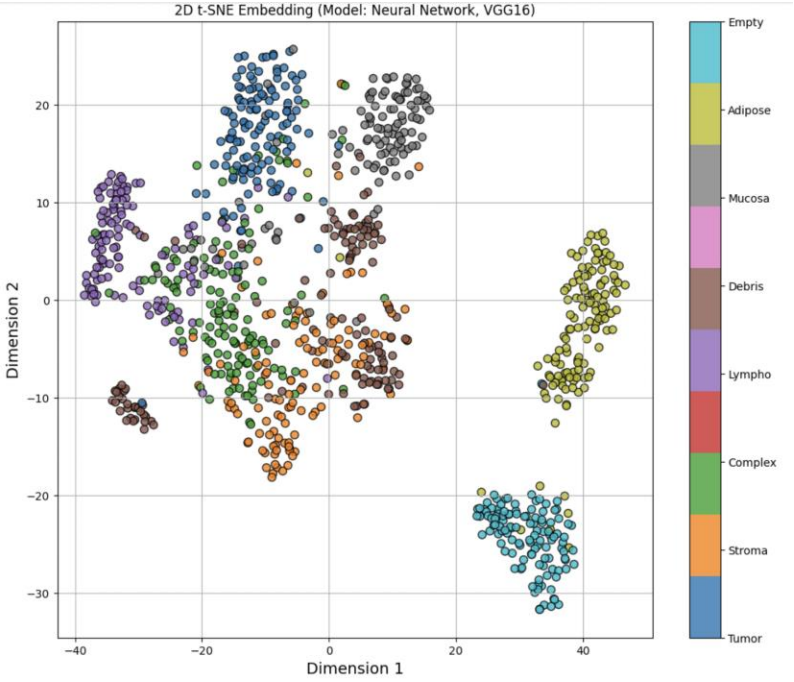
(*overfitting, but good results)

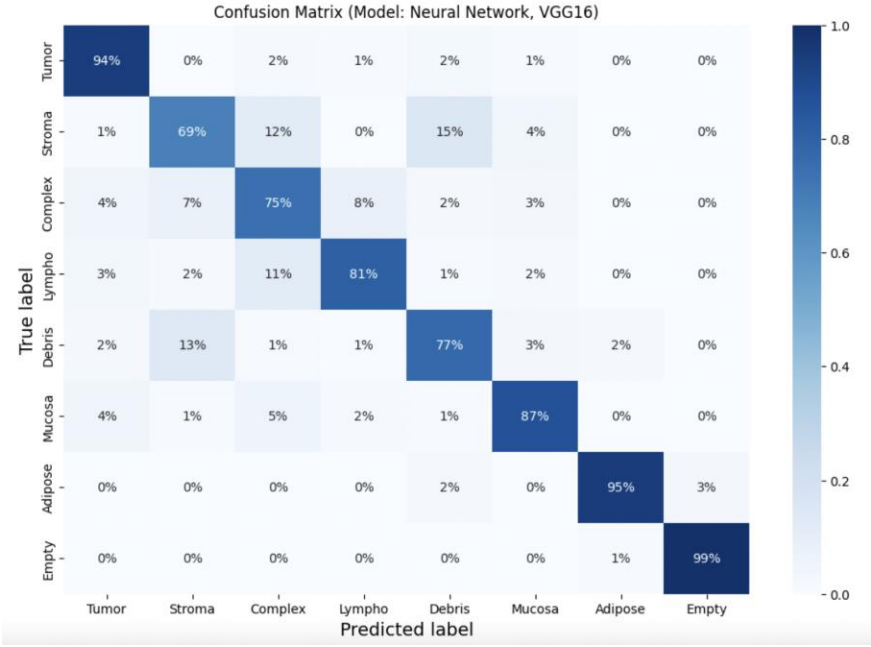


- Dimensionality reduction: VGG16 and then flattening, resulting in a 8192-D feature space.
- Classifier: Softmax classifier (epochs = 50, batch_size = 64).

```
(4000, 8192)
(1000, 8192)
```

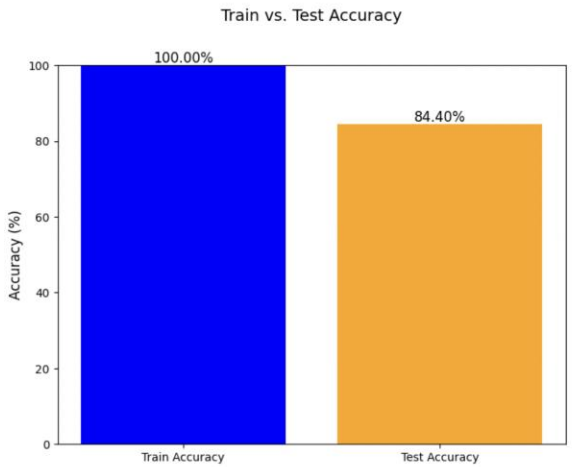
Model 9 – Neural Network + VGG16






```
Train Accuracy: 100.00%
Test Accuracy: 84.40%
```

(*overfitting, but good results)



- Dimensionality reduction: VGG16 and then flattening, resulting in a 8192-D feature space.

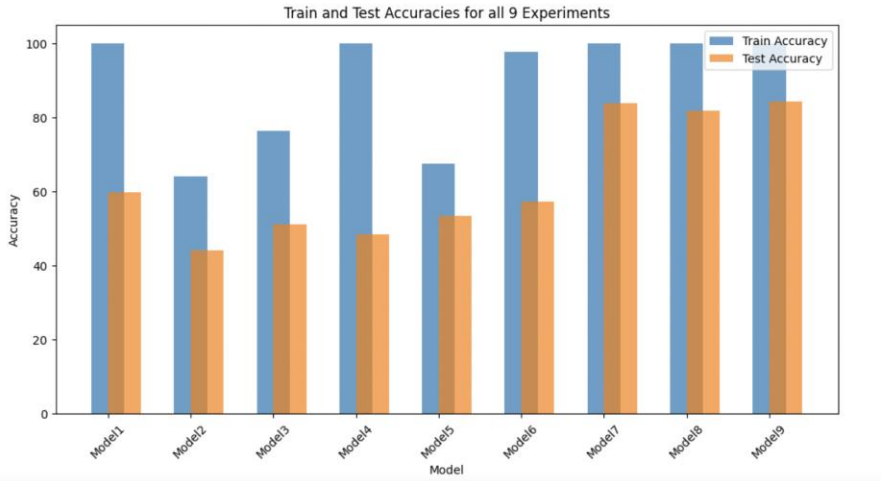
```
(4000, 8192)
(1000, 8192)
```

- Classifier: Neural Network classifier (epochs = 50, batch_size = 64).

Summary

To sum up, models that utilized VGG16 preprocessing demonstrated the effectiveness of this resizing method in reducing overfitting and improving test accuracy compared to downsampling or PCA. **Model 9, which combined a neural network with VGG16, stood out as the best model for our data**, displaying minimal overfitting and excellent performance on both training and test data. This model had the best clustering in the t-SNE plot as well.

	Method	Model	Train Accuracy	Test Accuracy
0	svm + downsample	Model1	100.000000	59.700000
1	softmax + downsample	Model2	64.025000	44.000000
2	network + downsample	Model3	76.350000	51.100000
3	svm + pca	Model4	100.000000	48.500000
4	softmax + pca	Model5	67.600000	53.400000
5	network + pca	Model6	97.800000	57.200000
6	svm + vgg16	Model7	100.000000	83.800000
7	softmax + vgg16	Model8	100.000000	81.900000
8	network + vgg16	Model9	100.000000	84.400000



Some ideas that can be done to get higher scores are:

1. **Hyperparameter Tuning –** fine-tune parameters like learning rate and regularization strength for better model fit.

2. **Data Augmentation –** creating more training samples through data augmentation (this method is good for image data).

3. **Ensemble Learning:** combining predictions from multiple models for improved accuracy.

4. **Transfer Learning:** leverage transfer learning by fine-tuning pre-trained models (for example – the VGG16 that we used).